

ACTIVIDAD AUTÓNOMA N.º 07

Endpoints, DTO y pruebas de conectividad

Proyecto	PharmaMobile integrado con PharmaSoft
Estudiante	Luciana Gutierrez
Asignatura	Desarrollo de Aplicaciones Móviles
Ciclo y semestre	VI · 2026-2
Fecha	4 de octubre de 2026
Rama	feature/ktor-client-Gutierrez
Repositorio	github.com/LucianaGutierrez-049/PharmaMobile

Alcance de la entrega

El informe documenta los cinco endpoints de Producto, los DTO reales de PharmaMobile, el mapper, cinco escenarios de prueba y las evidencias verificables obtenidas en Windows/Android. La ejecución en iOS se declara no realizada porque requiere macOS y Xcode.

Documento elaborado a partir del código del proyecto, del backend PharmaSoft, de ejecuciones HTTP reales, de pruebas automatizadas y de capturas tomadas durante la verificación.

1. Resumen de verificación

Resultado consolidado del trabajo técnico y documental.

Elemento	Resultado
Backend	PharmaSoft local operativo en http://localhost:8080
Android Emulator	Consume http://10.0.2.2:8080/api/v1/ mediante Ktor Client
GET listado	200 OK; datos deserializados, mapeados y mostrados en Compose
CRUD REST	GET, GET 404, POST, PUT y DELETE ejecutados contra PharmaSoft
Pruebas automáticas	22 pruebas Android Host aprobadas; 4 corresponden a conectividad y manejo de errores
Compilación	:androidApp:assembleDebug completó correctamente
iOS	No ejecutado: el equipo Windows no dispone de macOS/Xcode

Conclusión

La integración Ktor GET funciona de extremo a extremo en Android. Los escenarios 404, timeout y JSON desconocido están cubiertos por pruebas automatizadas; la desconexión también cuenta con evidencia visual. Las capturas iOS quedan fuera de esta ejecución por limitación de plataforma.

2. Arquitectura implementada

Capa	Componente	Responsabilidad
Red	HttpClientFactory	ContentNegotiation JSON, Logging ALL, timeout de 15 000 ms y configuración ignoreUnknownKeys.
Datos	ProductoApi	Ejecuta GET productos con pagina y tamaño.
DTO	ProductoResponseDto / PaginaResponseDto	Representan el contrato JSON de PharmaSoft.
Mapper	ProductoResponseDto.toDomain()	Convierte el DTO al modelo Producto y marca el origen remoto.
Repositorio	ProductoRepositoryImpl	Expone productos de dominio y desacopla la UI de la red.
DI	Koin / AppModule	Proporciona URL base, motor, HttpClient, API, repositorio y ViewModel.
Presentación	ProductoViewModel / UiState / Compose	Gestiona carga, éxito, error, reintento y renderizado de la lista.

3. Catálogo de endpoints

Contrato real comprobado en ProductoController y mediante solicitudes al backend.

Versionado

La ruta contractual es /api/v1/productos. Aunque la metadata general de Springdoc puede mostrar v0, la aplicación consume correctamente la versión v1 declarada por el controlador.

GET /api/v1/productos

Aspecto	Detalle verificado
Finalidad	Listar productos de forma paginada.
Entrada	Query: pagina:Int=0, tamaño:Int=20, ordenarPor:String=id, direccion:String=asc. Orden válido: id, nombre, precio o stock.
Éxito	200 OK con PaginaResponseDTO<ProductoResponseDTO>.
Errores	400 por tipos inválidos; 409 por paginación u orden no permitido; 500 no controlado.
PharmaMobile	Implementado y consumido por la aplicación.

GET /api/v1/productos/{id}

Aspecto	Detalle verificado
Finalidad	Consultar un producto.
Entrada	Path id:Long obligatorio; sin body ni query.
Éxito	200 OK con ProductoResponseDTO.
Errores	400 si id no es numérico; 404 si no existe; 500 no controlado.
PharmaMobile	Documentado; no forma parte del flujo de listado actual.

POST /api/v1/productos

Aspecto	Detalle verificado
Finalidad	Crear un producto.
Entrada	Body ProductoRequestDTO: nombre, precio, stock, estado y categoriaId.
Éxito	201 Created con ProductoResponseDTO.
Errores	400 por validación/JSON; 404 categoría; 409 nombre duplicado o integridad; 500.
PharmaMobile	Endpoint verificado; creación aún no implementada en la UI móvil.

PUT /api/v1/productos/{id}

Aspecto	Detalle verificado
Finalidad	Actualizar un producto completo.
Entrada	Path id:Long y body ProductoRequestDTO completo.
Éxito	200 OK con ProductoResponseDTO actualizado.
Errores	400 por validación; 404 producto/categoría; 409 duplicidad o integridad; 500.
PharmaMobile	Endpoint verificado; edición aún no implementada en la UI móvil.

DELETE /api/v1/productos/{id}

Aspecto	Detalle verificado
Finalidad	Dar de baja lógicamente un producto.
Entrada	Path id:Long; sin body.
Éxito	204 No Content; estado pasa a false.
Errores	400 id inválido; 404 inexistente; 409 ya inactivo; 500.
PharmaMobile	Endpoint verificado; baja aún no implementada en la UI móvil.

Ejecución real del CRUD

Solicitud	Resultado observado el 04/10/2026
GET /productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc	200 OK
GET /productos/999999999	404 Not Found
POST /productos	201 Created · producto de evidencia ID 21
PUT /productos/21	200 OK · precio 11.25 y stock 18
DELETE /productos/21	204 No Content · comprobación posterior estado=false

4. Diccionario de DTO y mapper

Campos, nulabilidad, valores predeterminados y correspondencia con el dominio.

ProductoResponseDto

Campo JSON	Tipo Kotlin	Obligatorio	Default	Nulable	Destino
id	Long	Sí	—	No	Producto.id
nombre	String	Sí	—	No	Producto.nombre
precio	Double	Sí	—	No	Producto.precio
stock	Int	Sí	—	No	Producto.stock
estado	Boolean	No	true	No	Producto.activo
categoriald	Long?	No	null	Sí	No transferido
categoriaNombre	String?	No	null	Sí	Producto.categoria
fechaCreacion	String?	No	null	Sí	No transferido
fechaModificacion	String?	No	null	Sí	No transferido

PaginaResponseDto<ProductoResponseDto>

Campo JSON	Tipo Kotlin	Obligatorio	Nulable	Uso
contenido	List<ProductoResponseDto>	Sí	No	Cada elemento se transforma en Producto
pagina	Int	Sí	No	Metadato de paginación
tamano	Int	Sí	No	Metadato de paginación
totalElementos	Long	Sí	No	Metadato de paginación
totalPaginas	Int	Sí	No	Metadato de paginación
ultima	Boolean	Sí	No	Indica última página

Reglas del mapper

ProductoResponseDto.toDomain() transfiere id, nombre, precio, stock, estado y categoriaNombre. Si categoriaNombre es nulo usa «Sin categoría». También fija stockDisponible=true y origen=REMOTO. categoriald, fechaCreacion y fechaModificacion no forman parte del modelo de dominio actual.

Configuración JSON final

```
JSON { ignoreUnknownKeys = true; isLenient = true }
```

La configuración permite evolucionar el backend agregando campos sin romper clientes anteriores. Los campos obligatorios sin default siguen protegiendo la estructura mínima del contrato.

5. Bitácora de pruebas de conexión

Cinco escenarios solicitados, con resultado y evidencia disponible.

ID	Escenario	Esperado	Observado	Estado
P1	GET exitoso	200, lista visible	Backend 200, Logcat y lista Android	COMPLETO
P2	Recurso inexistente	404 y mensaje controlado	Backend real 404 y test automatizado aprobado	VERIFICADO
P3	Sin conexión	UI estable y Reintentar	Mensaje de conexión visible; no se cerró	COMPLETO
P4	Timeout de 1 ms	HttpRequestTimeoutException controlada	MockEngine confirmó mensaje de timeout	VERIFICADO
P5	Campo JSON desconocido	Se ignora con configuración final	Modo tolerante y modo estricto aprobados	VERIFICADO

P1 · Respuesta exitosa

Se inició PharmaSoft, se abrió Productos en Android y se recargó la pantalla. Ktor registró REQUEST y RESPONSE 200. La UI mostró los registros del backend. Conclusión: integración de extremo a extremo aprobada.

P2 · 404

La solicitud real GET /productos/999999999 devolvió 404. ConexionRestTest comprobó ClientRequestException y el mensaje «No se encontró el producto solicitado.». La aplicación mantiene un UiState de error controlado.

P3 · Sin conexión

Con el servicio no accesible, Android mostró «No pudimos cargar el inventario» y «No se pudo conectar con el servicio. Verifica tu conexión a Internet.», además del botón Reintentar.

P4 · Timeout

Un cliente de prueba con requestTimeoutMillis=1 y MockEngine retardado produjo HttpRequestTimeoutException. El mapper devolvió «La conexión tardó demasiado. Verifica tu Internet e inténtalo nuevamente.». La configuración productiva quedó restaurada en 15 000 ms.

P5 · JSON desconocido

Con ignoreUnknownKeys=true, un campo adicional fue tolerado. Con un cliente estricto temporal, la deserialización falló y fue convertida a «No se pudo interpretar la respuesta del servidor.». No se alteró la configuración final de la aplicación.

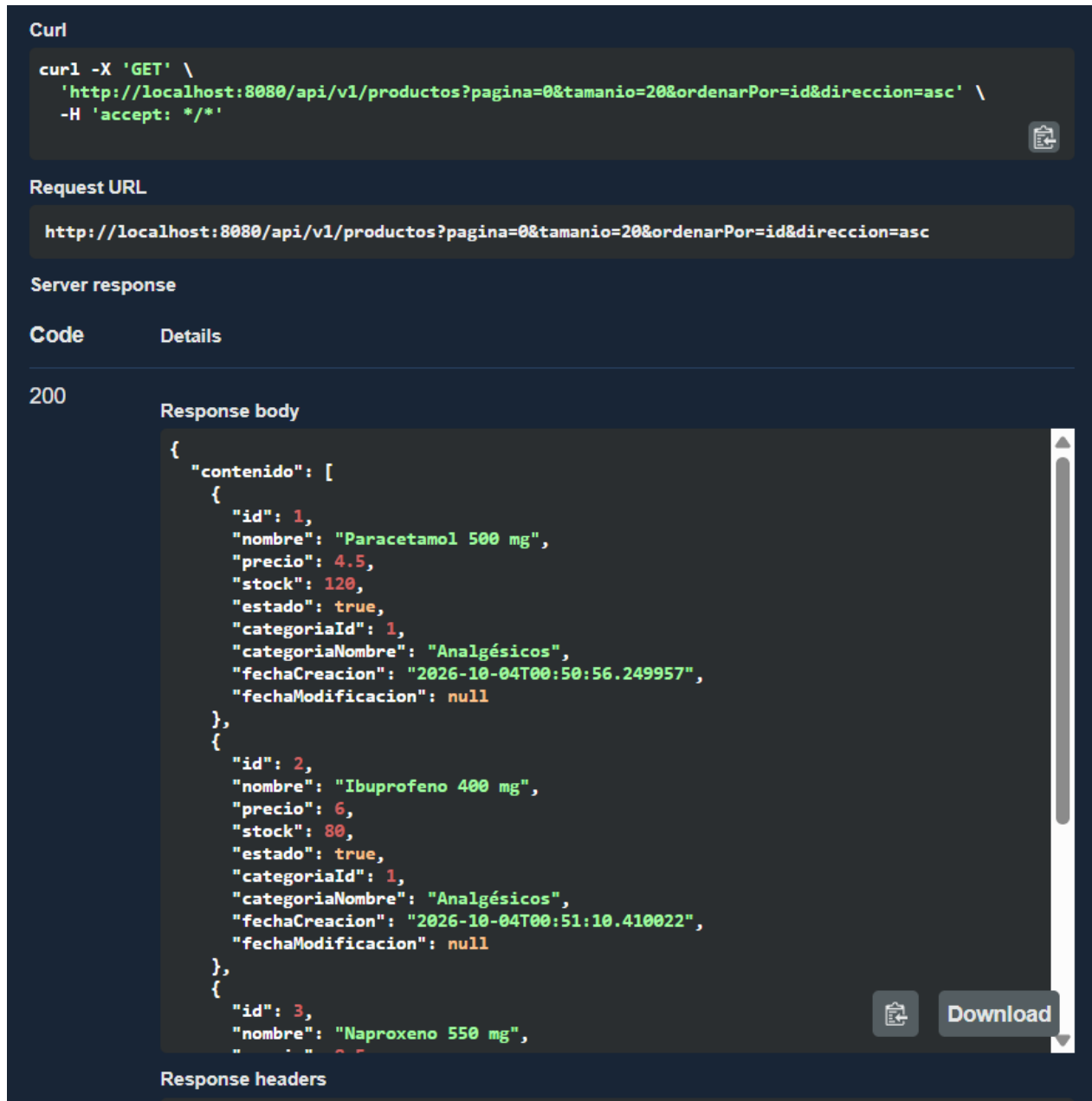
Resultado automatizado

:shared:testAndroidHostTest ejecutó 22 pruebas sin fallos. ConexionRestTest contiene cuatro pruebas específicas: 404, timeout y las dos variantes de campo desconocido. :androidApp:assembleDebug también terminó correctamente.

6. Evidencias de ejecución

Las imágenes siguientes corresponden a ejecuciones reales conservadas dentro del repositorio.

Evidencia 1 · Swagger GET 200 y respuesta JSON



The screenshot displays the Swagger UI interface for a REST API. It shows a GET request to the endpoint `http://localhost:8080/api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc` with a status code of 200. The response body is a JSON array of three product objects. The first object is for Paracetamol 500 mg, the second for Ibuprofeno 400 mg, and the third for Naproxeno 550 mg. Each object includes fields for id, nombre, precio, stock, estado, categoriaId, categoriaNombre, fechaCreacion, and fechaModificacion. The interface also shows the curl command used to execute the request and a 'Download' button for the response body.

```
curl -X 'GET' \
  'http://localhost:8080/api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc' \
  -H 'accept: */*'

Request URL
http://localhost:8080/api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc

Server response

Code    Details
200
Response body
{
  "contenido": [
    {
      "id": 1,
      "nombre": "Paracetamol 500 mg",
      "precio": 4.5,
      "stock": 120,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:50:56.249957",
      "fechaModificacion": null
    },
    {
      "id": 2,
      "nombre": "Ibuprofeno 400 mg",
      "precio": 6,
      "stock": 80,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:51:10.410022",
      "fechaModificacion": null
    },
    {
      "id": 3,
      "nombre": "Naproxeno 550 mg",
      "precio": 8,
      "stock": 50,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:51:10.410022",
      "fechaModificacion": null
    }
  ]
}
```

Response headers

Swagger ejecutado contra `http://localhost:8080/api/v1/productos`. Se observan Request URL, código 200 y productos devueltos por PharmaSoft.

6. Evidencias de ejecución (continuación)

Evidencia 2 · Ktor Client en Logcat



Registro extraído del emulador mediante ADB: GET a 10.0.2.2:8080 y RESPONSE 200.

Trazabilidad de red

Dato	Valor
Origen	Android Emulator Medium_Phone
Cliente	Ktor HttpClient
URL	http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
Respuesta	HTTP 200
Configuración final	Logging ALL · timeout 15 000 ms · ignoreUnknownKeys=true

6. Evidencias de ejecución (continuación)

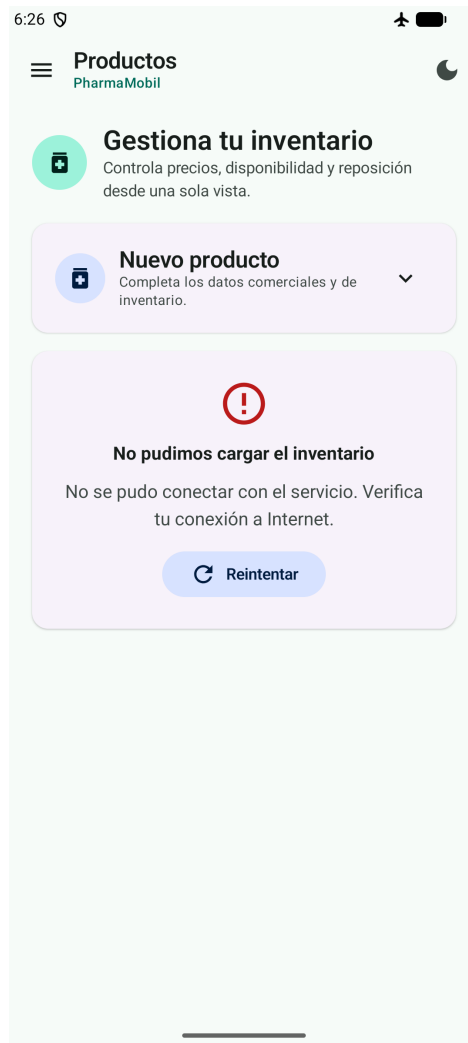
Evidencia 3 · Lista de productos en Android



La interfaz Compose muestra los productos recibidos de PharmaSoft, incluido el registro de evidencia utilizado para comprobar POST y PUT.

6. Evidencias de ejecución (continuación)

Evidencia 4 · Error sin conexión y recuperación

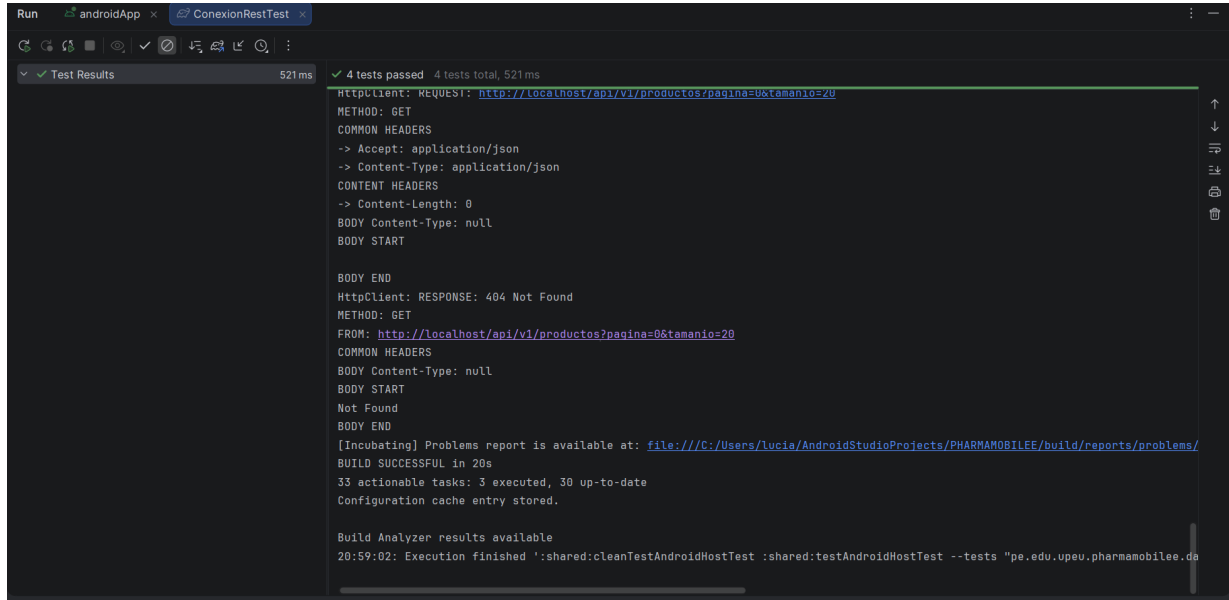


La UI conserva estabilidad, explica el problema y ofrece el botón Reintentar.

6. Evidencias de ejecución (continuación)

Evidencia 5 · Pruebas automatizadas de conectividad

Ejecución de ConexionRestTest desde Android Studio. La suite finalizó correctamente con 4 tests passed y BUILD SUCCESSFUL. Esta evidencia respalda los escenarios de recurso inexistente (404), timeout y campos JSON desconocidos.



The screenshot shows the Android Studio interface with the 'Run' tab selected. The console displays the following output:

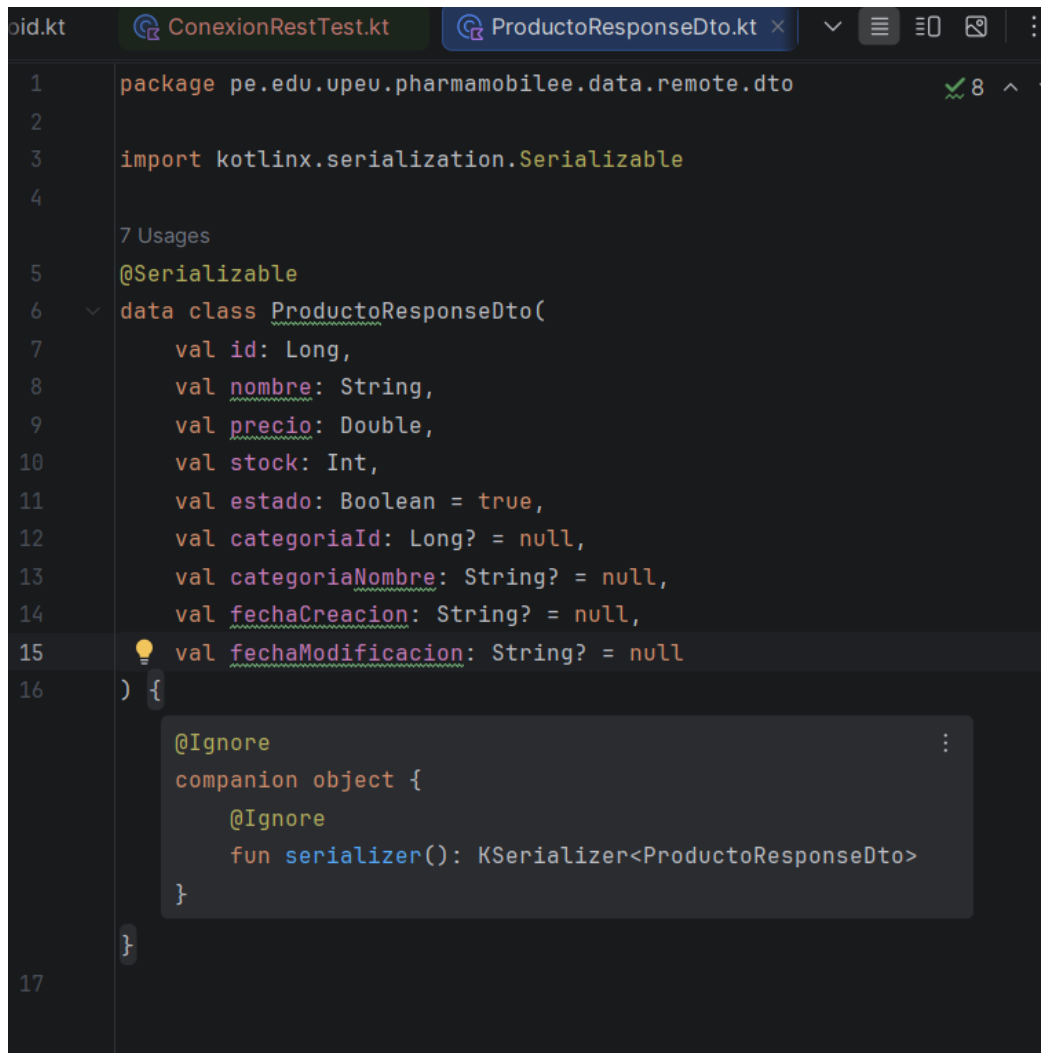
```
Run androidApp x ConexionRestTest x
Test Results 521 ms
4 tests passed 4 tests total, 521 ms
HttpClient: REQUEST: http://localhost/api/v1/productos/pagina=0&tamanio=20
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
BODY Content-Type: null
BODY START
BODY END
HttpClient: RESPONSE: 404 Not Found
METHOD: GET
FROM: http://localhost/api/v1/productos?pagina=0&tamanio=20
COMMON HEADERS
BODY Content-Type: null
BODY START
Not Found
BODY END
[Incubating] Problems report is available at: file:///C:/Users/Lucia/AndroidStudioProjects/PHARMAMOBILEE/build/reports/problems/
BUILD SUCCESSFUL in 20s
33 actionable tasks: 3 executed, 30 up-to-date
Configuration cache entry stored.
Build Analyzer results available
20:59:02: Execution finished ':shared:cleanTestAndroidHostTest :shared:testAndroidHostTest --tests "pe.edu.upeu.pharmamobilee.da'
```

Resultado de ConexionRestTest: cuatro pruebas aprobadas y compilación exitosa.

6. Evidencias de ejecución (continuación)

Evidencia 6 · Código real de ProductoResponseDto

El DTO usado para deserializar productos está declarado con `@Serializable`. La captura permite contrastar el contrato JSON con los tipos Kotlin, la nulabilidad y los valores predeterminados reales.



```
1 package pe.edu.upeu.pharmamobilee.data.remote.dto
2
3 import kotlinx.serialization.Serializable
4
5 7 Usages
6 @Serializable
7 data class ProductoResponseDto(
8     val id: Long,
9     val nombre: String,
10    val precio: Double,
11    val stock: Int,
12    val estado: Boolean = true,
13    val categoriaId: Long? = null,
14    val categoriaNombre: String? = null,
15    val fechaCreacion: String? = null,
16    val fechaModificacion: String? = null
17 ) {
18     @Ignore
19     companion object {
20         @Ignore
21         fun serializer(): KSerializer<ProductoResponseDto>
22     }
23 }
```

ProductoResponseDto.kt con los nueve campos documentados en el diccionario de DTO.

6. Evidencias de ejecución (continuación)

Evidencia 7 · Recurso inexistente: respuesta 404 real

Swagger ejecutó GET /api/v1/productos/{id} con el identificador 999999999. PharmaSoft respondió 404 Not Found y devolvió el mensaje de producto no encontrado, confirmando el escenario P2 con el backend real.

The screenshot displays the Swagger UI interface for a REST API. At the top, the method is set to GET and the URL is /api/v1/productos/{id}. A parameter 'id' is defined as an integer (\$int64) and is required. The value '999999999' is entered in the input field. Below the parameters, there are 'Execute' and 'Clear' buttons. The 'Responses' section is expanded, showing the response for status 404. The response body is a JSON object with the following structure:

```
{  "timestamp": "2026-10-04T21:31:03.9980498",  "status": 404,  "error": "Not Found",  "message": "Producto no encontrado con id: 999999999",  "path": "/api/v1/productos/999999999",  "validationErrors": null}
```

The response headers are also visible:

```
connection: keep-alive
content-type: application/json
date: Mon, 05 Oct 2026 02:31:04 GMT
```

Swagger: GET de producto inexistente con código HTTP 404 y respuesta JSON del backend.

7. Entregables y trazabilidad Git

Entregable	Ubicación / estado
Informe PDF	docs/S07_ActividadAutonoma_Gutierrez.pdf
Documentación técnica	docs/actividad_autonoma_sesion07/README.md
Capturas	docs/evidencias_sesion07 y docs/actividad_autonoma_sesion07
Pruebas	shared/src/commonTest/.../ConexionRestTest.kt
README del proyecto	Sección Conectividad REST actualizada
Rama	feature/ktor-client-Gutierrez
Repositorio	https://github.com/LucianaGutierrez-049/PharmaMobile
Commit técnico	55c54c2 · test: validar errores y compatibilidad JSON de Ktor
Commit documental	b900072 · docs: agregar evidencias de la actividad autonoma 07
Publicación	origin/feature/ktor-client-Gutierrez

8. Consideración de plataforma

iOS

No se ejecutaron pruebas ni se incluyeron capturas iOS porque el equipo disponible utiliza Windows y el simulador iOS requiere macOS con Xcode. Esta limitación se declara expresamente para mantener la autenticidad de la evidencia.

La cobertura funcional solicitada sí fue verificada en Android y mediante pruebas comunes de Kotlin Multiplatform. Para agregar evidencia iOS auténtica será necesario abrir iosApp/iosApp.xcodeproj en una Mac, ejecutar PharmaSoft y capturar el estado exitoso y un estado de error.

9. Checklist final

Criterio	Resultado
Cinco endpoints documentados con entradas, salidas y errores	CUMPLE
DTO completos, nulabilidad, defaults y mapper	CUMPLE
Cinco escenarios documentados y verificados	CUMPLE
Captura Swagger 200 + JSON	CUMPLE
Captura Ktor REQUEST/RESPONSE	CUMPLE
Captura Android de éxito	CUMPLE
Captura Android sin conexión	CUMPLE
Pruebas 404, timeout y JSON desconocido	CUMPLE mediante tests automatizados
README Conectividad REST	CUMPLE
Capturas iOS	NO APLICA EN ESTE EQUIPO WINDOWS
Commits y rama feature	CUMPLE · 55c54c2 y b900072